

How it all works: LLMs, GPUs, kernels, and carmen

A from-scratch guide to what carmen is building, in order: what a language model computes, what a GPU is good at, what a kernel is, why some kernels are slow, and how carmen writes and checks them. Numbers come from our own runs on an Apple M4 unless a source is cited.

1. What a language model does

A language model does one thing, over and over: **given the text so far, predict the next token.**

A **token** is a chunk of text, usually a word or part of a word. "There is a man who is" might be 6 tokens. The model has a fixed **vocabulary** of all the tokens it knows (Qwen2.5 has 151,936).

To write a sentence, it predicts one token, appends it, and repeats:

```
"There is a man who is"      → predict → "tall"  
"There is a man who is tall" → predict → "and"  
...
```

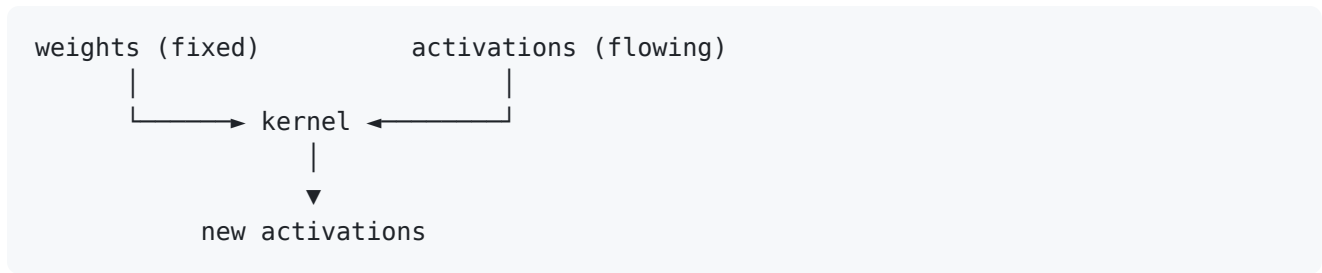
That loop is called **autoregressive generation**, and it's why writing is done one token at a time.

2. Three words you need: weights, activations, kernels

Weights are the model's fixed knowledge: huge tables of numbers learned in training. Qwen2.5-0.5B has about 0.5 billion of them. They never change while the model runs.

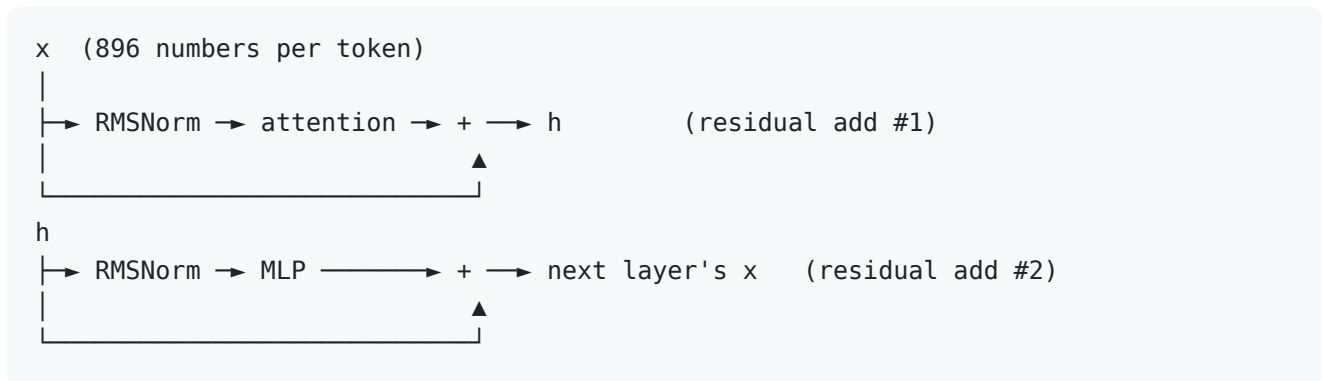
Activations are the model's working state for a given piece of text. Each token becomes a list of numbers (Qwen 0.5B: **896** numbers per token, its *hidden size*). As that list passes through the model, each layer changes it. Think of it as the model's evolving "thought" about that token.

Kernels are the small GPU programs that do each step: "multiply this activation by that weight table", "normalize this list", "add these two lists". A model is a long chain of kernels.



3. Inside one layer (Qwen2.5-0.5B)

Qwen2.5-0.5B: 24 layers, hidden size 896, 14 attention heads of size 64, 2 key/value heads, MLP size 4864 ([Qwen2.5 technical report](#)). Every layer has the same structure:



After 24 layers: a final RMSNorm, then a matrix multiply against the whole vocabulary gives one score per possible next token. Softmax turns scores into probabilities, and one token is picked.

3.1 RMSNorm: keep the numbers in a sane range

Over 24 layers, numbers could drift huge or tiny. RMSNorm rescales each token's list to a typical size of about 1 ([Zhang & Sennrich 2019](#)):

$$\text{RMSNorm}(x)_i = \frac{x_i}{\sqrt{\frac{1}{n} \sum_j x_j^2 + \epsilon}} \cdot w_i$$

- $n = 896$, w is a learned weight per position, ϵ is a tiny number (Qwen: 10^{-6} , Llama: 10^{-5}) so a list of all zeros doesn't divide by zero.
- Example: $x = [3, -4]$ gives $\text{RMS} = \sqrt{(9 + 16) / 2} = 3.54$, so $x / \text{RMS} = [0.85, -1.13]$.

LayerNorm, the older version, also subtracts the mean first: $\frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} w_i + b_i$. RMSNorm skips the mean and the bias: cheaper, and it works as well, so most new models use it.

3.2 The residual add: never throw information away

$$h = x + \text{attention}(\text{RMSNorm}(x))$$

Each block **adds** its result to the running activation instead of replacing it. Earlier information survives, and training works much better (He et al. 2015). The running activation is often called the **residual stream**.

3.3 Attention: which earlier tokens matter to me?

From each token's normalized activation, three matrix multiplies make a **query** q , a **key** k and a **value** v (Vaswani et al. 2017). For token i , with d = head size (64):

$$s_{ij} = \frac{q_i \cdot k_j}{\sqrt{d}} \quad p_{ij} = \frac{e^{s_{ij}}}{\sum_j e^{s_{ij}}} \quad o_i = \sum_j p_{ij} v_j$$

- s_{ij} : how relevant token j is to token i .
- p_{ij} : softmax turns relevance into percentages that add to 100%.
- o_i : a mix of the earlier tokens' values, weighted by relevance.

In "There is a man who is", the last "is" might give "man" 55% and "who" 30%: it pulls in "the subject is a man, and this is a description clause".

Causal mask. A token may only look at itself and earlier tokens, never the future: $p_{ij} = 0$ for $j > i$ (implemented by setting $s_{ij} = -\infty$). With a KV cache of L_k keys and L_q new queries, query i may see keys $j \leq i + (L_k - L_q)$.

Heads. Attention runs 14 times in parallel on different 64-number slices (14 **heads**), so different heads can track different relationships.

Grouped-query attention (GQA). Qwen 0.5B has 14 query heads but only 2 key/value heads; each K/V head is shared by 7 query heads (Ainslie et al. 2023). That's 7× less K/V to store and read.

RoPE. Attention by itself has no idea of word order. Rotary position embeddings rotate each pair of numbers in q and k by an angle proportional to the token's position, so the dot product $q_i \cdot k_j$ depends on the *distance* $i - j$ (Su et al. 2021).

KV cache. When writing token 101, the keys and values of tokens 1–100 haven't changed, so they're saved in a **KV cache** and only the new token's q, k, v are computed. That's why writing a token needs only **1 row** of new work.

3.4 The MLP: what do I know about this?

$$\text{MLP}(x) = W_{\text{down}}(\text{SiLU}(W_{\text{gate}} x) \odot W_{\text{up}} x)$$

- $W_{\text{gate}}, W_{\text{up}}$: $896 \rightarrow 4864$; W_{down} : $4864 \rightarrow 896$.
- $\text{SiLU}(z) = z \cdot \sigma(z)$, a smooth on/off switch; $\odot$ is element-by-element multiplication. This "gated" form is **SwiGLU** (Shazeer 2020).
- This is where most of the stored knowledge lives, and most of the weights.

3.5 Count the kernels

Per layer, roughly: 2 norms, 3 projections (q, k, v), RoPE, a cache write, attention, an output projection, 2 residual adds, 3 MLP matmuls, SiLU × multiply. That's **~12 kernels per layer × 24 layers ≈ 300 kernels per token**. Keep that number in mind.

4. Numbers in a computer: fp32, fp16, bf16, int4

A floating-point number stores a sign, an **exponent** (range) and a **mantissa** (precision).

format	bits	exponent / mantissa	largest value	precision	typical use
fp32	32	8 / 23	$\sim 3.4 \times 10^{38}$	~7 digits	math inside kernels
fp16	16	5 / 10	65,504	~3 digits	activations; can overflow
bf16	16	8 / 7	$\sim 3.4 \times 10^{38}$	~2–3 digits	activations in most new models: fp32's range, less precision (Google , bfloat16)
int8 / int4	8 / 4	whole numbers × a scale	n/a	rough	weights only (quantization)

Three rules that matter for kernels:

1. **Store small, compute big.** Activations travel as 16-bit to save memory traffic, but sums are done in fp32 inside the kernel, because adding thousands of fp16 numbers loses precision fast. carmen's judge checks this.
2. **Every result is rounded.** A correct kernel is never bit-identical to an exact answer; "correct" means "within a few roundings". carmen compares against a float64 answer key with a tolerance derived from the format, not picked by hand.
3. **fp16 overflows.** $300^2 = 90,000 > 65,504$. A kernel that squares in fp16 produces infinity.

Quantization stores weights in 4 bits: groups of 64 weights share one scale and offset (MLX's default), so each weight is a small integer 0–15, and real value $\approx \text{scale} \times q + \text{offset}$. A 0.5B model shrinks from ~1 GB to ~0.3 GB. Only weights are quantized; activations stay 16-bit.

5. What a GPU is good at

A CPU has a few fast, clever cores. A GPU has thousands of simple ones doing the same instruction on different data at the same time.

Metal (Apple's GPU language) organizes them like this:

GPU

├─ threadgroup a team that shares a fast scratchpad (32 KB threadgroup memory)
| └─ SIMD group 32 threads that move in lockstep; can combine values in one inst
| | └─ thread one worker; its own registers (the fastest storage)

memory	size	speed	shared by
registers	tiny	fastest	one thread
threadgroup memory	32 KB	very fast	one threadgroup
device memory (RAM)	GBs	slow (~100 GB/s on an M4)	everyone

The single most important fact: **arithmetic is cheap, moving data is expensive**. An M4 GPU can do several trillion arithmetic operations per second, but read only ~100 billion bytes per second.

5.1 The roofline: what limits a kernel

Every kernel is limited by one of two things ([Williams, Waterman & Patterson 2009](#)):

- **memory-bound**: it does little math per byte, so $\text{time} \approx \text{bytes moved} \div \text{bandwidth}$.
- **compute-bound**: it does lots of math per byte, so $\text{time} \approx \text{operations} \div \text{peak arithmetic rate}$.

The ratio is the **arithmetic intensity**: operations per byte. A norm does a couple of operations per number it reads, so it's memory-bound. A big matrix multiply reuses each number many times, so it's compute-bound. A good explainer of this way of thinking: [Horace He, "Making Deep Learning Go Brrrr From First Principles"](#).

There's a third limit the roofline leaves out, and it matters a lot for small models: **overhead**. Starting a kernel costs a few microseconds no matter how little work it does. On our M4, a norm on 896 numbers takes ~14–28 μs , while moving 896 numbers takes ~0.02 μs . That kernel is ~99.9% overhead.

6. What a kernel is, concretely

A kernel is a function that **every thread runs at once**, each on its own piece of the data. Our softmax kernel, simplified:

```

uint row = threadgroup_position_in_grid.x;    // one threadgroup per row
uint tid = thread_position_in_threadgroup.x;  // my index within the team

float m = -INFINITY;                          // 1. max of the row
for (int i = tid; i < n; i += TG) m = max(m, x[row*n + i]);
m = simd_max(m); /* ...then combine across SIMD groups via threadgroup memory... */

float s = 0;                                  // 2. sum of exp(x - max)
for (int i = tid; i < n; i += TG) s += exp(x[row*n + i] - m);
/* ...combine... */

for (int i = tid; i < n; i += TG)             // 3. write the answer
    out[row*n + i] = exp(x[row*n + i] - m) / s;

```

- `i = tid; i += TG` : thread 0 takes elements 0, 256, 512..., thread 1 takes 1, 257..., so neighbouring threads read neighbouring memory, which the hardware serves in one gulp (**coalesced** access).
- `simd_max` / `simd_sum` : 32 threads combine values in one instruction (a **reduction**).
- **Barriers** (`threadgroup_barrier`) make every thread wait until the whole team arrives. Forget one and threads read a value before it's written: a **race condition**, sometimes right and sometimes wrong.

Where Metal and MLX fit

your Python:	<code>mx.softmax(x)</code>	MLX's Python API
MLX:	picks its Metal kernel	~ hand-written by Apple engineers
Metal kernel:	runs on the GPU	

MLX lets you plug in your own Metal kernel body with `mx.fast.metal_kernel` ([MLX docs: custom Metal kernels](#)). That's what carmen does: its kernels run inside normal MLX code, so a better kernel is a better MLX.

7. How kernels get faster

technique	idea	example from our runs
fusion	do several steps in one kernel so in-between results never touch memory, and fewer kernels start	masked softmax: 3 kernels → 1, 1.82×
memory access	read many numbers per instruction (vectorized loads), coalesce, keep reused data in registers or threadgroup memory	matmul: 0.6× → 0.93× after Claude fixed scalar loads
tiling	split a big matmul into tiles that fit fast memory and reuse each loaded number many times	all of Claude's matmul kernels
special hardware	Apple's matrix units (<code>simdgroup</code> 8×8 matrix multiply)	matmul and attention kernels
better algorithms	one pass instead of three (online softmax)	attention
specialization	bake in exact sizes (896, head size 64): unroll loops, drop size checks	not yet: the next step for model kernels
tuning	try several tile and thread sizes, keep the fastest	built into carmen (up to 6 configs per kernel)
fewer launches	fusion, or "megakernels" covering many steps	the Qwen target

7.1 Fusion, with numbers

`softmax(x * scale + mask)` in plain MLX is three kernels:

```
kernel 1: read x          → write t1 = x * scale
kernel 2: read t1, mask   → write t2 = t1 + mask
kernel 3: read t2         → write out = softmax(t2)
total: 7 full passes over memory
```

Fused: read x and mask, compute everything in registers, write out: **3 passes**. Memory-bound, so the ideal speedup is $7/3 \approx 2.3\times$; carmen's kernel got 1.82×. Against `mx.compile` (which merges kernels 1 and 2), it's 5 vs 3 and carmen got 1.38×.

7.2 Tiling a matmul

$C = AB$ with A of size $M \times K$, B of size $K \times N$. Done naively, each output reads a full row of A and a full column of B from slow memory. Tiled: a threadgroup loads a 32×32 block of A and of B into threadgroup memory once, and every thread reuses it 32 times. Reads drop $\sim 32\times$, and the kernel becomes compute-bound, as a matmul should be.

7.3 Online softmax: the trick behind FlashAttention

Unfused attention writes the whole $L_q \times L_k$ score table to memory: for 4,096 tokens that's 16.7 million numbers per head. FlashAttention never writes it (Dao et al. 2022). It streams keys in blocks and keeps, per query, a running max m , running sum ℓ and running output $\tilde{o}$ (Milakov & Gimelshein 2018). For each new block of scores s_j :

$$m' = \max(m, \max_j s_j), \quad c = e^{m-m'}$$

$$\ell' = c\ell + \sum_j e^{s_j - m'}, \quad \tilde{o}' = c\tilde{o} + \sum_j e^{s_j - m'} v_j$$

At the end $o = \tilde{o} / \ell$. Multiplying by c re-bases old work to the new max, because $e^{s-m} e^{m-m'} = e^{s-m'}$, so the result is exact. Forgetting c is a classic bug that only shows when a later block has a bigger max, which is why carmen tests large, peaked scores.

8. Why "it runs and looks right" is not enough

GPU kernels fail in quiet ways:

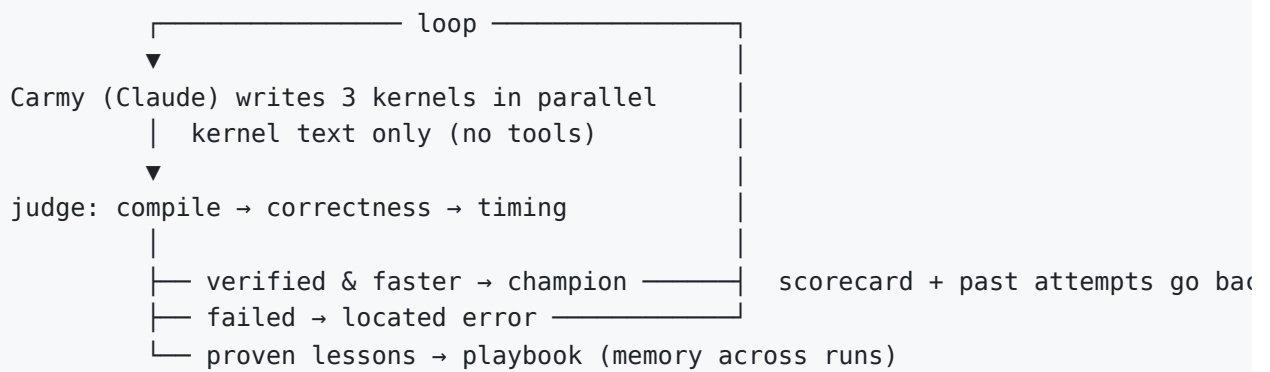
failure	what goes wrong	how you'd miss it
tail bug	the last few elements of a row aren't handled	every test size is a multiple of 256
precision	sums in fp16 drift	tolerance too loose
overflow	<code>exp(1000)</code> = infinity	test inputs are small
race	missing barrier	passes 99 runs in 100
square-only indexing	uses N where K belongs	test matrices are square
cache offset	causal mask ignores $L_k - L_q$	tests use $L_q = L_k$
unwritten output	some outputs never written	memory happened to hold the right values

Benchmarks for AI-written kernels (e.g. KernelBench) usually check one input shape with a loose tolerance. *Measuring the Checker* (Du et al.) measured how much that misses. In our own test, a one-shape check accepted **22 of 41** planted bugs at tolerance 1e-2, and 12 even at 1e-4.

Timing is just as easy to get wrong. Our first judge charged every kernel for an extra memory fill the stock op never pays: softmax looked 0.66× until we fixed it (0.95×).

9. What carmen is

carmen is a harness where an LLM writes Metal kernels and a deterministic judge decides what's real.



The parts:

- **Op spec** (`carmen/ops/`): what a kernel must compute. A float64 answer key, the exact contract (layout, edge cases, eps), trap inputs where bugs live, the stock MLX baseline, and the launch shape. Carmy never picks the launch grid, which removes a class of bugs.
- **Carmy** (`carmen/carmy.py`): the model call. It gets the contract, the chip, the current best kernel, the judge's feedback and a list of everything already tried. It returns kernel text and nothing else.
- **The judge** (`carmen/judge/`): runs in a separate process so a crashing kernel can't take down the harness.
- **Correctness**: hundreds of inputs (odd lengths, huge values, NaN/-inf rows, fp32/fp16/bf16), fresh random inputs every round, and a **secret draw** of shapes and data the model never sees. Outputs are pre-filled with NaN so unwritten outputs show up; a guard zone after the output catches writes past the end; the largest inputs run 5× and must match byte for byte (races).
- **Tolerance**: $\min(\max(4 \cdot \text{err}_{\text{MLX}}, \text{floor}), 16 \cdot \text{floor})$ with $\text{floor} = 2\epsilon_{\text{dtype}} + 8\epsilon_{\text{fp32}} \sqrt{n}$: output rounding plus fp32 accumulation over n terms, measured relative to each row's scale. Fixed in advance, never tuned per kernel.
- **Speed**: interleaved timing against stock MLX and `mx.compile`, a bootstrap confidence interval, and % of measured memory bandwidth. Faster than the memory system allows is flagged as suspicious.
- It says **where** a kernel is wrong ("tail bug, fails at n=33"), not just that it is.
- **The loop** (`carmen/loop.py`): the fastest verified kernel becomes the champion; the next round sees its per-shape scorecard and every past attempt. It stops near the bandwidth limit or when it stops improving. `--mode bon` runs blind best-of-N as a control.
- **Memory** (`carmen/memory.py`): lessons are kept only if the judge's verdicts back them, not because the model said they worked.
- **Measuring the judge** (`carmen/broken`): planted bugs per kernel; the judge must catch every one and accept the reference kernel.
- **Profiling a real model** (`carmen/profile`): loads a model with `mlx-lm` and times every step of a layer at its real shapes, so we know what's worth a kernel.

10. What we found

kernel	what MLX does	carmen's best	vs <code>mx.compile</code>	sizes it never saw
masked_softmax	3 kernels	1.82×	1.38×	1.72×
add_rmsnorm	2 kernels	1.41×	1.43×	1.47×
softmax / layernorm / rmsnorm	1 tuned kernel	0.95–0.99×		
matmul	1 heavily tuned kernel	0.93×	0.92×	0.86×
fused attention	1 heavily tuned kernel	0.54×	0.51×	0.79×

1. **AI-written kernels don't beat hand-tuned code; they beat code nobody fused.**
2. **The judge works:** 41/41 planted bugs caught across four kernels; a one-shape check missed 22.
3. **On hard kernels, Claude wasn't wrong, it was slower.** All 28 matmul and attention kernels that compiled were correct.
4. **Feedback helped where there was a clear bottleneck** (matmul 0.6× → 0.93×, in two separate runs) and barely elsewhere. On simple kernels the first draft is already near the ceiling.
5. **The judge has to be fair, not just strict.** It wrongly rejected 12 good kernels for `#pragma unroll`, and one planted bug survived because a trap input wasn't extreme enough. Both were found and fixed.

11. Real models: where the time goes

From `carmen profile` on the M4, writing one token (decode, batch 1):

	Qwen2.5-0.5B	Llama-3.2-1B
decode speed	~155 tok/s	106 tok/s
time in matmuls	63%	81%
time in small steps (norms, adds, RoPE, SiLU, attention)	37%	19%

Reading a 512-token prompt (prefill) is 91–96% matmul for both.

The speed limit for writing

To write one token, the model must read every weight once:

$$\text{max tokens/s} \approx \frac{\text{memory bandwidth}}{\text{bytes of weights}}$$

model (4-bit)	weights	limit at ~96 GB/s	measured	share of the limit
Qwen2.5-0.5B	~0.3 GB	~300 tok/s	~155	~50%
Llama-3.2-1B	~0.75 GB	~128 tok/s	106	~83%

So:

- **Small models are slow because of overhead** (~300 tiny kernels per token). Fusing steps, so fewer kernels run, is the lever. That's the current carmen target: one kernel for "residual add + RMSNorm" that returns both results, used twice per layer, turns 4 small kernels into 2.
- **Bigger models are close to the physical limit** for writing, because MLX's 4-bit matmul is already well tuned. There the levers are reading fewer bytes per token ([speculative decoding](#), a quantized KV cache), long-context attention that reads each shared K/V head once, and prefill, which is compute-bound.
- **Kernels matter most where nobody has written a good one yet**: new architectures (mixture-of-experts, state-space models, new attention variants), new quantization formats, and specific shapes. That's carmen's real use: write the missing kernel quickly, and prove it correct.

12. Glossary

term	meaning
token	a chunk of text the model reads and writes
hidden size	numbers per token in the activation (896 for Qwen 0.5B)
activation	the model's working state for a token, changed by each layer
weight	a learned, fixed number; the model's knowledge
layer	one repeat of norm → attention → add → norm → MLP → add
residual stream	the running activation that each block adds to
head	one of several parallel attention computations on a slice of the activation
KV cache	stored keys and values of earlier tokens, so they aren't recomputed
GQA	several query heads sharing one key/value head
prefill	processing the prompt (many tokens at once, compute-bound)
decode	writing tokens one at a time (memory- or overhead-bound)
quantization	storing weights in fewer bits (4-bit) with a scale per group
kernel	a GPU program for one step, run by thousands of threads at once
threadgroup	a team of threads sharing fast memory
SIMD group	32 threads moving in lockstep
reduction	combining many values into one (sum, max)
barrier	wait until every thread in the team gets here
coalesced access	neighbouring threads reading neighbouring memory
fusion	doing several steps in one kernel
tiling	splitting work into blocks that fit fast memory
memory-bound / compute-bound	limited by moving data / by arithmetic
launch overhead	fixed cost of starting a kernel, however small its work
roofline	the model of which limit applies to a kernel
tolerance	how far from the exact answer a correct result may be

13. Further reading

- [Vaswani et al., *Attention Is All You Need* \(2017\)](#): the transformer.

- [Horace He, *Making Deep Learning Go Brrrr From First Principles*](#): compute vs memory vs overhead. The single best read for this project.
- [Dao et al., *FlashAttention* \(2022\)](#) and [Milakov & Gimelshein, *Online normalizer calculation for softmax* \(2018\)](#): fused attention and the online-softmax trick.
- [Williams, Waterman & Patterson, *Roofline* \(2009\)](#): memory-bound vs compute-bound.
- [Zhang & Sennrich, *RMSNorm* \(2019\)](#) · [Su et al., *RoPE* \(2021\)](#) · [Ainslie et al., *GQA* \(2023\)](#) · [Shazeer, *GLU variants / SwiGLU* \(2020\)](#): the building blocks of a Qwen/Llama layer.
- [Qwen2.5 technical report \(2024\)](#): the model we profile.
- [Leviathan et al., *Speculative decoding* \(2022\)](#): writing faster on big models without new kernels.
- [MLX: custom Metal kernels](#) and Apple's *Metal Shading Language Specification*: how carmen's kernels plug in.
- [Ouyang et al., *KernelBench* \(2025\)](#) and [Du et al., *Measuring the Checker*](#): how AI-written kernels are evaluated, and where that evaluation falls short.